**Hackathon website : https://codexhackathon.sea.com/**

# Ho so Ai

## Problem

Administrative procedures take far more time and effort than they should. Renewing a passport, registering a business, changing residence: each one means working out which documents apply to your case, tracking down the right forms, filling them in correctly, and hoping you have not missed something that will send you back a second time.

Most of that work is mechanical. What makes it slow is that the rules sit across national, ministry and local sources written for the officials processing the file rather than the person preparing it, the forms give no guidance on how they should be completed, and nothing is carried over from one procedure to the next even though the same personal details are needed every time.

People respond the way anyone would. They put it off, or they pay someone else to handle it.

The goal is to make these procedures simple. One place where a person says what they need to do, and gets back a clear list of what applies to their situation and completed forms ready to submit.

## Target Users

Anyone in Vietnam dealing with paperwork. Citizens and residents renewing a passport, getting a new ID card, registering a business, changing address, registering a birth. Foreigners and tourists renewing a visa or applying for a work permit.

We care most about people doing it for the first time, who have no family member to ask and no money to hire an agent. They are the ones who lose the most time to a single missing document.

Ward and district staff benefit too, since they spend their days turning away files that are incomplete for reasons nobody ever told the applicant.

## Solution

One place for any Vietnamese administrative procedure, where every answer comes from the official source.

You ask in your own words. "I need a new passport." "I want to register my online shop." The assistant finds the procedure in the national database, asks a few quick questions to work out which situation you are in, and then shows you the whole task at once. Every document you personally need, in what order, how long it takes, what it costs, and the mistakes that usually get files rejected. Each requirement links back to the official rule it came from, so you can check it yourself instead of taking our word for it.

Then it does the paperwork with you. You enter your personal details once and they are reused for every procedure after that. When you open a document, you see the real form on screen with its fields listed beside it in plain language, so it is obvious what is being asked and where it goes. For official PDFs with proper form fields we fill them directly. For the ones that are only scanned images we use a vision model to find where each field sits on the page. Either way you review everything before anything is generated, and what you download is the real document, formatted correctly and ready to hand in. Nothing is ever submitted on your behalf.

The difference between this and asking a chatbot is that we never answer from memory. If the official source does not say it, we tell you we do not know.

**Codex**

**I plan to use Codex as an active pair-programmer throughout the hackathon, not just for boilerplate. Concretely: (1) Scaffolding fast generating the initial project structure, API routes, and data models so the team can get to a working skeleton within the first hour instead of losing time on setup. (2) Speeding up agent orchestration logic since building agentic workflows (tool calling, multi-step reasoning, state management) involves a lot of repetitive but detail-sensitive code, I'll use Codex to draft these components quickly, then review and adjust the logic myself rather than accepting it blindly, since agent code is exactly the kind of thing that looks right but fails silently if not checked. (3) Debugging under time pressure  pointing Codex at error traces or unexpected outputs to get fast hypotheses, especially useful in a hackathon where we can't afford to get stuck for 30 minutes on a stack trace. (4) Writing tests and edge cases quickly asking Codex to generate test scenarios for the core logic so we can move fast without breaking things right before a demo. (5) Documentation on the fly having Codex draft a clear README and setup instructions in parallel with development, so the judges and other teams can actually understand and try what we built, instead of that being an afterthought at the last minute.**

**My approach is to treat Codex as a way to move faster on the parts that are well-understood and repetitive, so the team can spend our limited time on the parts that actually require judgment: the architecture decisions, the reliability of the agent's outputs, and making sure the demo tells a clear story.**